Theory of Algorithms – Greedy Techniques

Patrick Marais
Department of Computer Science
University of Cape Town
patrick@cs.uct.ac.za





Introduction

- Greedy technique repeatedly tries to **maximize** the return based on examining **local conditions**, with the hope that the outcome will lead to a desired outcome for the **global problem**
- Examples:
  - Dijkstra's single-source shortest paths
  - Minimum spanning tree construction – Prim & Kruskal
  - Change making
  - Huffman coding

Principle

- Optimization problems solved through a sequence of choices that are:
  - Feasible – satisfy problem constraints
  - Locally optimal – best choice among all feasible options for that step
  - Irrevocable – no backing out
- A greedy grab of the best alternative, hoping that sequence of locally optimal steps will lead to a globally optimal solution
- Sometimes approximation is acceptable (local optimality waived)
- Not all optimizations can be solved this way
- Methods like induction can be used to prove optimality in some cases; more complex cases use abstract mathematics e.g. "Matroids"



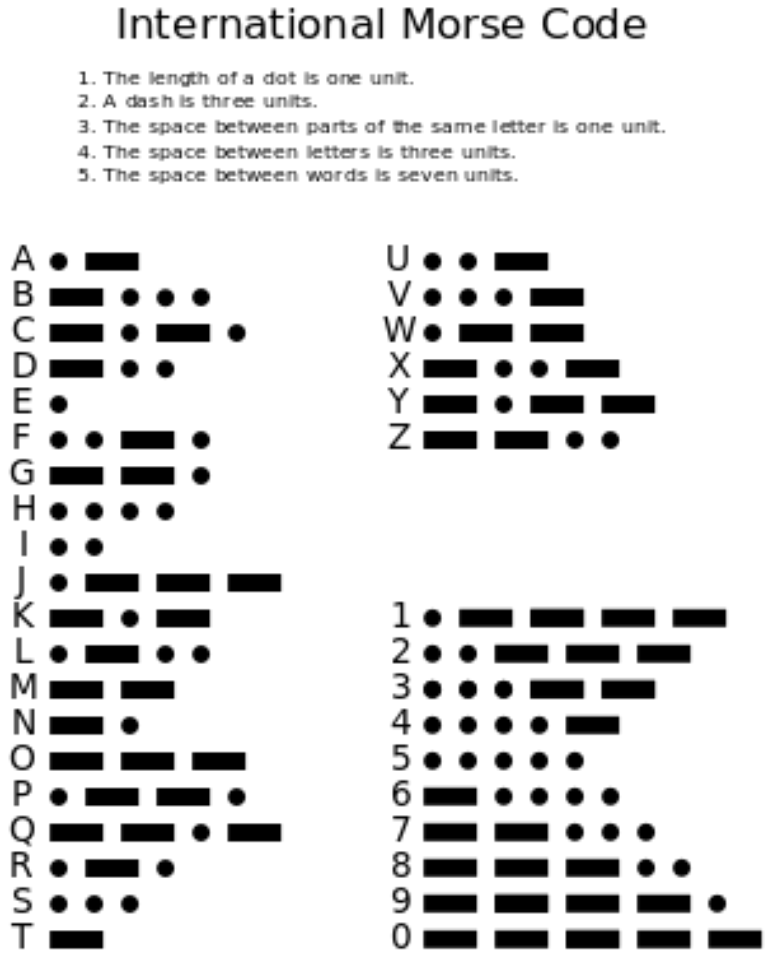
Change Making Revisited

- Problem:
  - Give change for a specific amount n, with the least number of coins of the denominations $d_1 > d_2 > \ldots > d_m$
- Example:
  - Smallest change for R2.54 using R5, R2, R1, 50c, 20c, 10c, 5c, 2c, 1c
  - R2 + 50c + 2c + 2c = R2.54
- Algorithm:
  - At any step choose the coin of the largest denomination that doesn't exceed the remaining total; Repeat
  - Optimal for reasonable sets of coins
  - Think of an example where it isn't optimal?



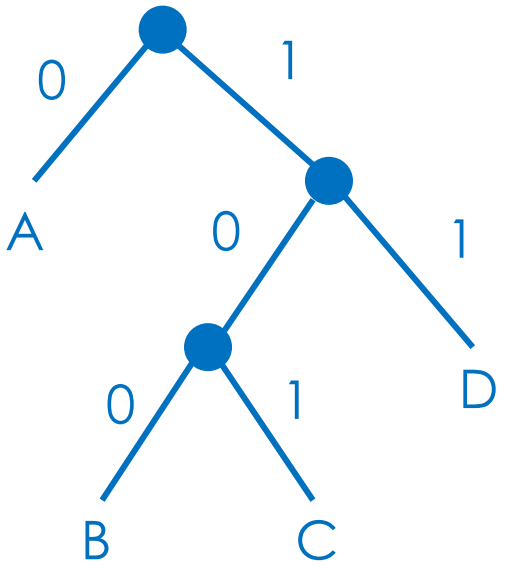
Text Encoding

- Assign bit patterns to an alphabet of characters so that in general texts fewer bits are used overall (entropy coding)
- Fixed length
  - Same number of bits for each character
  - E.g., ASCII
- Variable length
  - Number of bits for a character varies according to probability of occurrence
  - Leads to better compression
  - e.g., Morse's telegram codes



Huffman Trees

- How to determine which bits belong to which character in a variable length encoding
- Solved by Prefix-free codes where no code is the prefix of another character
- Huffman Tree:
  - Implements prefix-free codes
  - Leaves are characters, left edges encode 0-bits, right edges 1-bits
  - Walk the tree to encode and decode
  - Example encoding: ABADC → 0100011101
  - Example decoding: 11000101 → DAAAC

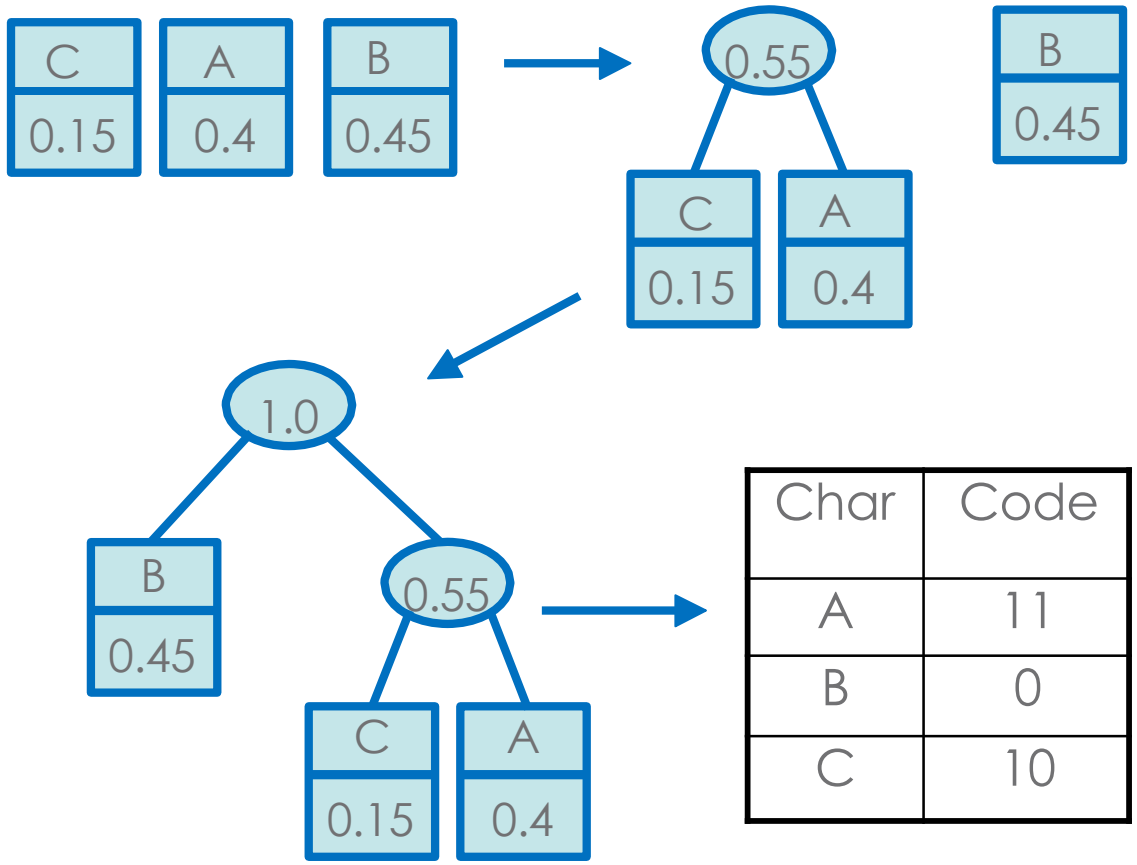


Huffman Coding Algorithm

- Build a Huffman Tree given an alphabet and associated probabilities

1. Initialize *n* one-node trees labelled with the characters of the alphabet
2. Record the frequency/weight of the character in the root
3. REPEAT until a single tree is obtained:
   - Find two trees with the smallest weight
   - Make them the left and right sub-tree of a new tree and record the sum of their weights in the root

Huffman Coding Example



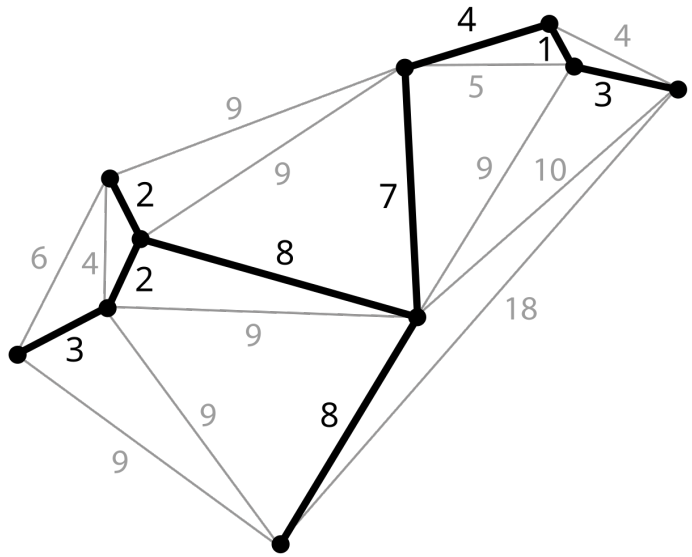
Char	Code
A	11
B	0
C	10

Notes on Huffman Coding

- Compression Ratio:
  - Standard measure of compression
  - *CR = 100 * (y - x) / y*, where x is compressed and y is uncompressed
  - Typically, 20-80% in the case of Huffman Coding
- Yields optimal compression provided:
  - Probabilities of character occurrences are independent
  - and are known in advance and negative powers of 2 e.g. $\frac{1}{2^3}$
  - BUT terrible with bit strings: alphabet = {0,1}. Why?
- More sophisticated approaches – Adaptive Huffman, builds frequency table on the fly

Minimum spanning tree (MST)

- Spanning tree of an undirected connected graph is a tree of connected nodes such that all graph vertices are contained once
- A minimum spanning tree is a spanning tree of "minimal weight" (sum of weights) for a graph with edge weights
  - Used in network communications, social media (and other) relationships, approx. solution to TSP etc
- Exhaustive construction is exponential – instead, we use a greedy approach. Two different greedy algorithms:
  - Prim, adding vertices - O(|E| log |V|)
  - Kruskal, adding collection of edges - O(|E| log |E|)
- Both provide provably optimal MSTs



Source: wikipedia

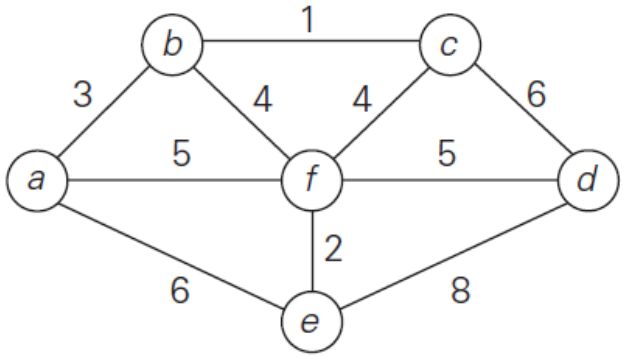
Prim's Algorithm

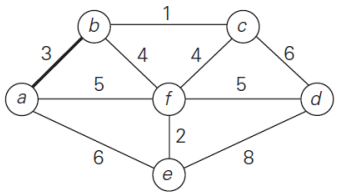
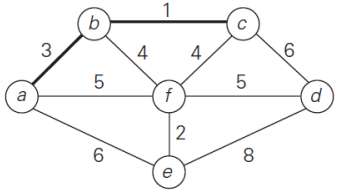
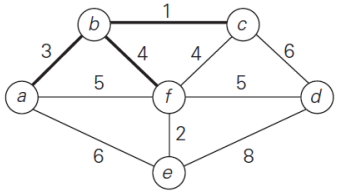
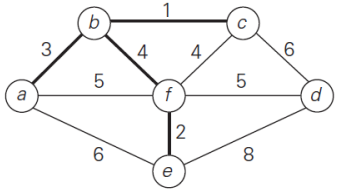
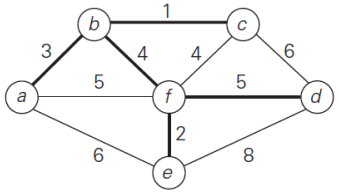
- Greedy construction of expanding subtree from initial vertex, v_1 chosen from vertex set V
- Initialize each remaining directly connected vertex with edge label/weight e.g
  - $v_2(v_1,3)$ – node v_2 connects to v_1 (already in our tree) with weight 3
  - For unconnected vertices, weight = ∞
- Add nearest vertex NOT in tree at each iteration
  - The vertex selected will have lowest edge weight when connected to tree (it can connect to ANY tree vertex)
  - After vertex, v*, is added, remove from list of unprocessed vertices, and update edge weights for vertices that remain
- Weight adjustment:
  - check nontree vertices that connect to v*
  - If they have a lower edge weight, change their 'label' to v* and their weight to the smaller weight we found
  - This requires finding minimum of set of values – use min-heap

ALGORITHM *Prim(G)*

//Prim's algorithm for constructing a minimum spanning tree
//Input: A weighted connected graph $G = \langle V, E \rangle$
//Output: E_T, the set of edges composing a minimum spanning tree of G
$V_T \leftarrow \{v_0\}$ //the set of tree vertices can be initialized with any vertex
$E_T \leftarrow \varnothing$
for $i \leftarrow 1$ **to** $|V| - 1$ **do**
  find a minimum-weight edge $e^* = (v^*, u^*)$ among all the edges (v, u)
  such that v is in V_T and u is in $V - V_T$
  $V_T \leftarrow V_T \cup \{u^*\}$
  $E_T \leftarrow E_T \cup \{e^*\}$
return E_T

Prim's Algorithm - Example



Tree vertices	Remaining vertices	Illustration
a(−, −)	**b(a, 3)** c(−, ∞) d(−, ∞) e(a, 6) f(a, 5)	
b(a, 3)	**c(b, 1)** d(−, ∞) e(a, 6) f(b, 4)	
c(b, 1)	d(c, 6) e(a, 6) **f(b, 4)**	
f(b, 4)	d(f, 5) **e(f, 2)**	
e(f, 2)	**d(f, 5)**	
d(f, 5)		

Prim's Algorithm - complexity

- Better for dense graphs compared to Kruskal's Algorithm
- Time complexity is $O(|E| \log |V|)$
  - using adjacency list rep, and
  - min-heap - which is $O(\log |V|)$ - for closest vertex lookup
- Can be optimized further
- Best choices depend on graph characteristics e.g.
  - Fibonacci heap reduces to $O(|E| + |V| \log |V|)$
  - For dense graphs $|E| = \Theta(|V|^2)$
  - But easier approaches too since Fibonacci is complex to implement

Greedy Algorithms - Summary

- Strengths:
  - Intuitively simple and appealing
- Weaknesses:
  - Only applicable to optimization problems
  - Doesn't always produce an optimal solution
  - More sophisticated alternatives work better – arithmetic coding

Course Conclusion

- Coverage:
  - Algorithmic strategies for solving computational problems
  - Including well-known solutions to common optimization, computational geometry and graph problems
- Key Skills:
  - Problem solving – Understand how to apply the different algorithmic strategies
  - Analysis – be able to determine the efficiency of a simple iterative or recursive algorithm
  - Knowledge and application – know the common algorithms under each strategy and be able to apply them to sample data